Journal of Software Engineering Research and Development
This work is licensed under a Creative Commons Attribution 4.0 International License.

HalluCodeDetection: Detecting Hallucinations in Code Generation

Rubens Nascimento de Lima

Abstract

Keywords: *Code Generation, Hallucination Detection, Specialized Fine-tuning, Small Language Models*

1 Introduction

Developers are increasingly using Large Language Models to write and review code (Lee et al., 2025; Mahade Hasan et al., 2026). Most of these models are so large that they can only run on external computers (Nguyen et al., 2026). This raises concerns about usage limits and cost, as well as keeping important information safe (Nguyen et al., 2026; Shen et al., 2025). On the other hand, Small Language Models are cheaper to run and can work on a local computer (Mahade Hasan et al., 2026; Shen et al., 2025). This helps reduce security risks and reduces dependence on external services (Nguyen et al., 2026; Shen et al., 2025). These small models aren't as good as the big ones at writing code (Mahade Hasan et al., 2026). So the code they create might look plausible. Behaves incorrect. This is called Code Hallucination (Pavel et al., 2025; Lee et al., 2025; Tian et al., 2025). It's also hard to find these mistakes (Andriushchenko et al., 2026; Lee et al., 2025) because the models don't understand how code works the way a real programmer does. They just learn how to write code by looking at lots of examples. So writing code isn't the same as knowing all the details and the problems it might have.

In this work, we use the term *Code Hallucination* to describe the phenomenon in which the model generates plausible but incorrect code, whose flaws translate into observable errors at an execution level (Pavel et al., 2025; Lee et al., 2025; Tian et al., 2025). Following previous work on errors and hallucinations in code generation (Lee et al., 2025; Dou et al., 2025; Tian et al., 2025), we define 3 error levels, each associated with an observable execution level.*Syntax Error* refers to hallucinations about the grammatical structure of the language that result in syntax errors that prevent compilation (Lee et al., 2025; Dou et al., 2025).*Runtime Error* is a hallucination in which the model makes mistakes in execution logic and specific code knowledge, leading to failures during execution (Lee et al., 2025; Dou et al., 2025).*Functional Error* is a hallucination that deviates from functional requirements and can be detected through manual checks or automated tests (Lee et al., 2025; Dou et al., 2025; Tian et al., 2025), and it can be dangerous because it can be subtle enough to go unnoticed in poorly controlled environments.

Our central idea, therefore, is to study SLMs not only as code generators, but also to evaluate their real ability to analyze code correctness. We investigated whether various SLMs can recognize whether code is well-written or contains errors, and the degree of those errors, including in code the model generates itself. We also investigated how different forms of targeted training can improve these models' ability for this task. We compared these models with static tools to understand how they can serve as a useful alternative in real software environments, identify flaws without execution, and how combining static tools with the models can improve both their ability to analyze code and to correct it. Finally, we compared SLMs with larger LLMs to understand the difference between models that can run locally on a home computer and those that run on supercomputers.

To conduct this investigation, we used the EvalPlus benchmark, which provides some basic Python programming tasks and runs on various LLMs, primarily SLMs. We analyzed their code capabilities and, based on the execution of the generated code, measured how often models generated code that passed all available test cases and the level of execution achieved. After collecting hallucination data, we used the models to generate evidence-based explanations of generation errors for model training, using a variation of target parameters and metrics. Finally, py_compile and pyright were used for a static analysis of syntax and execution errors.

The results highlight a divergence between error generation and error-detection capabilities. Most tested SLMs achieved 70-80% accuracy in code generation, surpassing the larger models used in this study. However, the main problem with all tested models in code generation is incomplete task understanding, which results in code gaps and missing logic needed to fully solve the problem. In error detection, no model was able to accurately detect all classes, especially the *Runtime Error* class, in which the model with the highest recall rate was 32%, and the median among the models was only 14.6%, highlighting the models' shortcomings in understanding code execution in practice. However, even with this limitation, the models still detect runtime failures better than static tools, showing they can detect runtime failures without execution and that combining them with static tools for manual compilation verification can make detection more robust. Finally, we found that specialization can substantially improve detection of these errors; however, the models may develop a pronounced bias due to class imbalance.

The main contributions of this work are:

- An empirical evaluation of SLMs for both code generation and error detection under a common execution-based reference, distinguishing Syntax, Runtime, and Functional Errors through multilabel classification.
- An analysis of *self-error detection*, investigating

whether models can recognize errors in code they generated themselves and examining the relationship between generation and evaluation capabilities.
- A specialized SLM-based detector trained with execution-derived labels and teacher-generated explanations, together with an analysis of how different optimization objectives affect the detector's behavior across error categories.
- An investigation of the relationship between model-based reasoning and deterministic static analysis, evaluating static techniques independently, as supporting evidence for error detection, and as feedback during iterative code generation.
- A comparison between SLMs and larger models, assessing whether a clear gap exists in error generation, error detection, and tool assistance, to determine whether SLMs can be a viable alternative for these tasks.

This work is divided into sections that initially review how the literature has addressed code hallucinations, error detection, and specialization methods. Next, it describes the methodology and experimental setup. The results section presents the main findings on the SLM's code generation and evaluation ability; error-detection specialization; comparison and use of static tools; and, finally, a comparison of these abilities with larger models. The article concludes by discussing factors that may affect the results, summarizing the main findings, and suggesting areas for future research.

2 Related Work

Pavel et al. (2025) characterizes hallucinations as seemingly plausible but factually incorrect or meaningless outputs that can manifest through syntax violations, invalid references, or conflicts with API knowledge, and divides them into 4 major groups: syntax, runtime, functional, and code quality hallucinations, as does the work of Lee et al. (2025). The work of Dou et al. (2025) reinforces using the first 3 classes to distinguish error types.

In our work, we distinguish the hallucination itself from its observable consequence. Instead of inferring the cause of an incorrect generation, we identify the hallucination type from its observable effect, dividing it into *Syntax Error*, *Execution Error*, or *Functional Error*. This results-oriented abstraction is intentionally more abstract than some previous taxonomies, which, in addition to presenting macro classes, also present subclasses. However, this classification enables broader analysis of the model's deficiency rather than reducing it to a single measure of correctness. Code quality issues are outside the scope of this study, since our analysis focuses on executability, not on non-functional requirements such as style, maintainability, and efficiency.

Empirical studies have already investigated how failures vary across LLM models, including minor variations. The previously cited error-classification work, Dou et al. (2025), also empirically evaluates, using subsets of the same benchmark employed in our article, combined across different sizes. The results showed that, regardless of size, LLMs' main difficulty is producing code that meets all functional requirements (Dou et al., 2025). On average, smaller models were 16.03 percentage points below larger models, mainly because larger models are better at meeting these functional requirements (Dou et al., 2025). These results provide a basis for our study, as they already describe a process for classifying error types and compare models of different sizes (Dou et al., 2025). Our work reuses the established division and also proposes new models to understand how the distance between larger and smaller models changes over time under the same set of test cases.

The detector-oriented work most closely related to ours is Andriushchenko et al. (2026), who construct line-level hallucination annotations and train a lightweight Transformer detector using internal representations from language models. Their results demonstrate that specialized detectors can identify hallucinated regions and outperform generic uncertainty-based approaches. Our work investigates a different granularity and setting. Rather than locating individual hallucinated lines, we ask whether general-purpose and specialized SLMs can classify the observable failure mode of a complete generated solution. We additionally study self-error detection, class-specific behavior, and the interaction between model-based detection and external software-analysis evidence.

3 Methodology

This study focuses on investigating the capabilities of SLMs across various lines of hallucination, from generating valid code to verifying code, and how factors such as specialization and static analysis assistance can impact the classification performed by these models, and finally, how these models compare to larger models in these tasks.

The main experimental factors in this study are the model evaluated, the specialization strategy, and the availability of feedback from static tools. We apply the same protocol across all analyses, using the same programming tasks, error taxonomy, prompt structures, automatic output extraction scheme, and tool-derived feedback.

For error classification, the model uses only the task description and target code, without labels or execution results. The classification is multi-label, since a code can exhibit more than one type of error; for example, a runtime error and, even if there were no runtime error, a functional error. In this analysis, we verified the model's ability to detect its own errors and those of other models.

The study is organized around seven research questions:

- **RQ1:** To what extent can SLMs produce valid code, and what are the main hallucination problems in the generated code?
- **RQ2:** To what extent can SLMs correctly identify errors in code?
- **RQ3:** To what extent can specialized SLMs improve error detection in code?
- **RQ4:** How do static analysis techniques compare to SLMs in detecting errors in generated code?
- **RQ5:** How does feedback from static analysis affect the ability of language models to detect errors in generated code?

- **RQ6:** How does an iterative feedback loop from static analysis affect the ability of language models to generate correct code?
- **RQ7:** How do SLMs compare to substantially larger language models in terms of code generation, error detection, and tool-assisted configurations?

3.1 Benchmark and Unit of Analysis

For this study, we used several models to generate code to perform a specific programming task. As a basis, we used the EvalPlus benchmark (Liu et al., 2023), which extends the test case sets of HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), totaling 563 programming tasks. We used all benchmark tasks uniformly, without separating them by repository subtype, since both have executable function-level tasks in Python and reinforced test sets under a common evaluation structure.

Each task includes a natural-language specification describing the required functionality, including algorithmic problems, numerical operations, and data structure manipulation. This treats each task as an independent unit of analysis. For each task, the models follow a protocol for the code output so that a test bench evaluates the generated code and, from its execution, determines the type of hallucination, if any. The results serve as evidence for comparing code-generation capabilities and as a data source the models will use to classify the code.

3.2 Ground Truth and Error Classification

In this work, *Code Hallucination* refers to the phenomenon in which a model produces plausible but incorrect code, observable at different execution levels, as shown in Pavel et al. (2025), Lee et al. (2025), and Dou et al. (2025).*Syntax Error* refers to hallucinations about the language's grammatical structure that produce syntax errors and prevent compilation.*Runtime Error* is a hallucination in which the model makes mistakes in execution logic and specific code knowledge, leading to failures during execution.*Functional Error* is a hallucination that deviates from functional requirements and can be detected through manual checks or automated tests, and it can be dangerous because it can be subtle enough to go unnoticed in poorly controlled environments.

Ground truth is determined from the compilation, execution, and result of the code in the established test suite, as presented in the Algorithm 1. Labels are accumulated across the test cases of a given task, allowing *Rundown Errors* and *Functional Errors* to occur simultaneously, depending on the test-case results.

Code with *Syntax Errors* is unique because, since the code fails at the start of execution, it is impossible to determine through execution whether other error levels exist in the code. An important point is that code marked as a syntax error should not always be treated as a deviation from the code's grammar; it can also be a deviation from the request, with the model deviating from the established output rule, providing an output that is outside the specification of the extraction made by the code. Although it is possible to manually retrieve the model's response in some of these cases, to maintain methodological rigor and remain on the same line of execution, we decided to mark this type of hallucination, since the definition of *Syntax Hallucination* refers to the model's difficulty with the language's grammar, and the model's output does not follow the requested grammar.

The compilable code is executed in different tests within the set, with a maximum execution time. Tests classified as *Runtime Error* are those where the model failed to understand the code, produced invalid elements or operations that return a specific error, or failed in the execution logic, causing the code to exceed the time limit. On the other hand, *Functional Error* refers to errors where the code completes execution, but the result differs from expectations, indicating the model failed to implement all functional requests.

Finally, code classified as *Correct* passes all requested test cases without errors. This is not a formal proof of correctness for all possible inputs, but only that, for the test set in question, all criteria are well met. Therefore, it does not overlap with the other classes. In other words, although our classification is multi-label, only the *Runtime Error* and *Functional Error* classes can have an overlap.

Algorithm 1 Execution-based ground-truth labeling procedure

Require: Generated candidate source and EvalPlus tests
Ensure: Set of outcome labels L

$L \leftarrow \emptyset$
if candidate does not compile **then**
    $L \leftarrow L \cup \{\text{Syntax Error}\}$
    return L
end if
for each test case **do**
    execute candidate
    if execution raises an exception or exceeds the timeout **then**
        $L \leftarrow L \cup \{\text{Runtime Error}\}$
    else if observed output $\neq$ expected output **then**
        $L \leftarrow L \cup \{\text{Functional Error}\}$
    end if
end for
if $L = \emptyset$ **then**
    $L \leftarrow \{\text{Correct}\}$
end if
return L

Table 1 summarizes the operational definitions used throughout the study.

3.3 Evaluation Metrics

To measure performance on the code generation task, we used the pass@1 metric, the proportion of tasks in which all tests were correct, relative to the total number of tasks. The other hallucination class metrics are the sets of tasks marked with errors, relative to the total number of tasks. The evaluation does not consider partial success; that is, even if a task has only one test case with an error, we treat the entire generated code as a hallucination because the model deviated from the task.

Table 1. Operational outcome categories used in the study

Outcome	Operational definition
Syntax Error	The candidate source cannot be successfully compiled after response normalization.
Runtime Error	The candidate compiles, but execution raises an exception, exceeds the timeout, or otherwise fails during a test case.
Functional Error	The candidate executes without a Runtime Error for a test case but produces an output different from the expected result.
Correct	The candidate compiles, produces no Runtime Error, and returns the expected result for every available test case.

For error detection, we independently analyzed how the model classified the code in each class. Because of this independence, a prediction may be correct for one class while simultaneously introducing an error in another. For example, if the reference contains only *Run-off Error* and the model predicts the classes *Run-off Error* and *Functional Error*, the prediction accounts for a true positive for *Run-off Error* and a false positive for *Functional Error*.

From the class-level analysis, we verified traditional metrics such as Precision, Recall, and F1 for each class. Recall is the most used metric in this work for analyzing each class, as it indicates the model's ability to identify real hallucination cases in that class. We mainly use the F1-score to balance precision and recall.

For a more global analysis, we used accuracy metrics calculated on the exact set; that is, in multi-label scenarios, accuracy occurs when all predicted and real classes match. However, because the dataset is highly unbalanced and to better evaluate class behavior overall, we also used macro metrics, such as Macro F1 and Macro Recall, which average the metrics across classes.

Finally, Self-error detection is a subset of the detection inferences performed, but attributed only to the models. These classes are interesting for observing how models detect their own hallucinations, but may have a problem when compared with others, because since they are not sets from others, they may have a different number of examples and difficulty in classifying classes; it only points out how well the model is able to detect flaws in its own code.

3.4 Statistical Analysis

We apply the statistical analyses used in this work to each research question, treating the programming task as the independent unit of analysis, and the procedures correspond to the conditions evaluated in these tasks. We apply inferential tests only when they directly support a comparison required by the respective research question.

For research question 3, we used a task-level paired bootstrap to verify whether specialization alters the ability to detect errors relative to the same model before fine-tuning, preserving task pairings and constructing a confidence interval for the observed difference. This technique uses several resamplings, paired to recalculate the difference between the specialized model and the base model.

For research question 6, we used a two-tailed McNemar exact test, which tests whether two conditions applied to the same units have the same probability of a positive result. To obtain this binary classification, we labeled all hallucination cases as Not Correct. This technique aims to verify whether the feedback protocol systematically alters the proportion of correct solutions compared to the initial protocol. If the null hypothesis holds, the changes should be proportional across groups. However, this does not allow us to conclude that the effect was caused solely by static analysis; it may be due to other variables, such as model iteration.

3.5 Specialized Error Detectors

To perform model specialization, we used a process based on execution-based labeling. As a complement, a model was used to generate the explanation of the labeling, in which, for syntax errors, it was indicated exactly where the error occurred; for execution and functionality errors, the model was informed of the test cases in which the model erred, with the expected output and the output obtained, in addition to the code execution errors, if any. In this way, the label is obtained deterministically, while the model provides the explanation.

The motivation for explaining the labeling is based on previous work. For example, (Hsieh et al., 2023) shows that using rationales provides an additional supervision signal for training smaller models, improving performance compared with label-only supervision.

To verify each model's ability to detect hallucinations, we divide the data into three subsets: training, validation, and testing. All comparisons between models are performed only on the test set, and the same applies to untrained models, to provide a fairer basis for comparison.

4 Experimental Setup

4.1 Models and Inference Configuration

The study evaluates 10 open-source SLMs ranging from 4 to 35 billion parameters and, to provide a comparative baseline with larger models, includes 2 larger LLMs. Table 2 presents an overview of all models used in this study, organized by size.

The SLMs used in these experiments used 4-bit quantization, enabling evaluation of larger local models within the available computational capacity. Previous empirical studies support the use of quantized models, such as (Giagnorio et al., 2026), which showed that 4-bit quantization can significantly reduce memory requirements while maintaining performance close to that of corresponding full-precision models. This justification lets us expand our search for more SLMs and avoid being limited by the computational constraints of this experiment.

During training, we used Glimmer 30B, Qwen 3.8 27B, and Ornith 1.5 35B to generate classification justifications. For model modification and specialization, we readjusted the Gemma 3 4B, Gemma 4 E2B, and Gemma 4 E4B models.

Table 2. Models evaluated in the study.

Model	Parameters
Small language models	
Devstral Small 2	24B (Rastogi et al., 2025) (Mistral AI, 2025)
GLM 4.7 Flash	30B (GLM-4.5 Team et al., 2025) (Z.ai, 2025)
Gemma 3 4B	4B (Gemma Team et al., 2025)
Gemma 4	{5.1, 8, 31}B (Gemma Team et al., 2026)
GPT-OSS 20B	20B (OpenAI et al., 2025)
Muse Glimmer 30B	30B (Meta Superintelligence Lab, 2026)
Ornith 1.5 35B	35B (Ornith Team, 2026)
Qwen 2.5 Coder	{7, 32}B (Hui et al., 2024)
Qwen 3 Coder 30B	30B (Yang et al., 2025) (Qwen Team, 2025)
Qwen 3.8 27B	27B (Qwen Team, 2026)
Large models used for comparison	
DeepSeek V4.1 Flash	552B (DeepSeek-AI, 2026)
MiMo V2.5	310B (Xiaomi MiMo Team, 2026)

The initial code generation experiment uses a temperature of 1.0. For all subsequent phases, we use a temperature of 0.0 to minimize sampling variability. In all evaluated conditions, support for internal Chain of Thought mechanisms was disabled.

4.2 Code Generation Experiment

In the code generation experiment, all 563 programming tasks from the EvalPlus benchmark are evaluated. All inferences include a system prompt instructing the model to act as a Python programming assistant and to return a single function implementation, depending on the task type. In HumanEval+, the model receives the function signature and is prompted to complete the function body. With MBPP+ tasks, the model is prompted to generate the complete function implementation. Models are instructed to respond without adding any supplementary information, such as explanations, tests, or usage examples.

Before execution, the response is normalized for insertion into the function used by the test suite. In responses containing Markdown text, only the sections marked as Python and comprising the function body are extracted. However, if a complete function of the same type is returned, the definition established by the model is used. In cases where the output is not entirely code, or when there is no Markdown markup indicating code, the normalization procedure does not remove arbitrary natural language content that appears before or after the generated code, which may cause a *Syntax Error*.

4.3 Error Detection and Self-Evaluation Experiment

For the code hallucination detection experiment, a system prompt is provided that defines the classification task and an explanation of the classes *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*. The model is instructed to return a structured JSON containing a dictionary where the keys are the hallucination classes and the values are natural-language justifications for choosing each class. These explanations should be general and not address specific cases. The code already applied to the previous normalization process is passed alongside the system prompt.

If a detector's response is malformed, we classify it as an empty structure. We adopted this methodology to prevent malformed responses from being excluded from the reported metrics and to avoid skewing the balance of the model's classifications. For example, if a model classifies only easy cases and does not respond to more complex ones, comparing only valid classifications will distort the metric. Although this approach favors sets with more negative cases, this problem is mitigated because one of the main evaluation metrics is Recall, which focuses on how many real cases were reported. Self-assessment, as a subset of evaluations, does not require an additional procedure.

4.4 Specialized Detector Training

4.4.1 Knowledge Distillation

To train the specialized detector, we use supervised learning from execution and result verification, along with compiler execution results and invalid test outputs. In correct cases, the model is instructed that the code in question is valid for the presented task. The evaluator is instructed to agree with the label and generate only the explanation of that label, without referring to specific edge cases.

Each generated solution is processed independently by different rationale-generating models, producing its own justification for the same code and reference label. This strategy aims to diversify the response explanations and reduce rationale association bias. The analyses do not evaluate the validity or quality of the rationales; they evaluate only the deterministic labels. In total, 18,579 records were produced from 6,193 generated solutions. As the The model generates only an explanation; it does not need to create a parser or produce structured output.

4.4.2 Dataset Partition and Class Distribution

The dataset is generally subdivided by programming task unit into training, validation, and test sets, with proportions of 65%, 15%, and 20% of the total tasks, respectively. This ensures that, during training, no explanation for a given output is generated by another model or by similar code. Table 3 summarizes the distribution of tasks and the number of explanations generated.

We divide the data using a grouped assignment process with a fixed random seed of 42. The seed orders the tasks, and a greedy algorithm attempts to divide them to preserve a stratified distribution of classes across all sets.

Table 3. Task-level partition of the rationale-expanded dataset.

Split	Tasks	Records
Training	366	12,078
Validation	84	2,772
Test	113	3,729
Total	563	18,579

Since the models showed positive results in code generation, most classifications belong to the class *Correct*, occurring in about 72.4% of records, while the class *Syntax Error*, which had the fewest cases, accounts for only 1.5%. It's worth noting that, because *Runtime Error* and *Functional Error* can co-occur, the sum of the three classes, in raw form, exceeds 100%.

The predominance of *Correct* results motivated class balancing in some training processes, where the training set underwent a reduction in classes with larger records to match a set with a fixed ratio to equalize the number of records with the classes with fewer records, such as 3:3:1 and 1:1:1. However, the validation and test records remain unchanged under all circumstances.

4.4.3 Fine-Tuning and Hyperparameter Search

During training, the models are trained using simulated detection inference cases. Each sample contains 3 messages: the first is a system prompt that defines the task, the output structure, and the 4 possible classifications. The user input is the task instruction and the respective code to be evaluated, already normalized. The third input is marked as the assistant's input; that is, the target to which the model must respond, already structured as an object with the classes labeled by the execution and the justification generated by the models.

To fit within the available computational resources, we train with QLoRA, 4-bit NF4 quantization, and low-rank adapters, following the LoRA approach.

For hyperparameter search, the parameters and values observed in Table 4 were used, including those specific to LoRA and other general Deep Learning training parameters. To train with the best possible configurations, we used Optuna (Akiba et al., 2019) with the Tree-structured Parzen Estimator sampler, multivariate sampling, and a linear-constant kernel. The search engine should converge to the group of tested hyperparameters that most closely approximates the target metric of the execution.

Table 4. Hyperparameters explored during specialized detector training.

Parameter	Values
LoRA rank (r)	{8, 16, 32}
LoRA alpha (α)	{8, 16, 32}
LoRA dropout	{0.05, 0.1}
Learning rate	{5e-5, 1e-4}
Bias	{none, all, lora_only}
Epochs	{3, 4}

The study evaluates different optimization objectives in two rounds. One target is accuracy, which aims to train the model to correctly predict more examples in the validation set. In the Macro-F1 evaluation, we select the outputs that achieve a better F1-score per class. However, during the training cycle, before calling Optina, since it's not possible to parse the output to evaluate the loss calculation, we calculate a weight for each class token and give greater weight to these tokens in the selection.

We also added other configurations, such as class balancing and excluding the class with the fewest records, which can be easily detected during static analysis. With these tools, the models achieved excellent performance on this class. The final model is selected based on the configuration that maximizes its corresponding selection metric on the validation partition. The test partition is used only after model selection for the final evaluation.

4.5 Static Analysis Configuration

To perform static analysis, we use checkers to detect errors without test cases. To investigate *Syntax Error*, we use the language's built-in compiler (Python Software Foundation, 2026), Pyright (version 1.1.414), for Python 3.13.15. This checker is the same one used at runtime, so it will always have the final verdict on whether the code has a syntax error, serving as an exact verification template.

To detect *Runtime Errors*, we use Pyright (Microsoft, 2026), which, although it is not a real template, attempts to identify static conditions associated with potential runtime failures that compilation alone cannot expose. In fact, even when we run the compilation test, we cannot match the compiler's rate, so we use this tool to predict this type of error. The classes checked by the analyzer are listed in Table 5.

Table 5. Pyright diagnostic groups mapped to potential Runtime Errors.

Diagnostic group	Examples of detected problems
Name and binding errors	Undefined, unbound, or possibly unbound variables.
Call and argument errors	Invalid function calls, incompatible arguments, or invalid call signatures.
Type consistency errors	Incompatible assignments, return values, general type mismatches, or invalid type arguments.
Attribute, indexing, and operator errors	Invalid member access, unsupported indexing, or invalid operand combinations.
Optional-value misuse	Calling, indexing, iterating over, or accessing members of values that may be *None*.

4.6 Tool-Assisted Experiments

4.6.1 Error Detection with Static Feedback

The error detection experiment with the static feedback assistant uses the same set of model responses for each of the 113 tasks in RQ2 to establish a controlled comparison of error detection based on the impact of the added content. The only variable added to this text is the static analysis feedback, while the task description, code, classification prompt, output scheme, temperature, and evaluation metrics remain unchanged.

For each sample, the model receives the same system-prompt instruction, and the output from the static tools, showing how the code was analyzed in each category, is added to the user prompt. It is also added to the prompt that the tools may provide incomplete or incorrect evidence. Figure 1 illustrates the flows. As in the general detection experiment, an unparseable detector response is represented as an empty prediction set and remains in the evaluation.

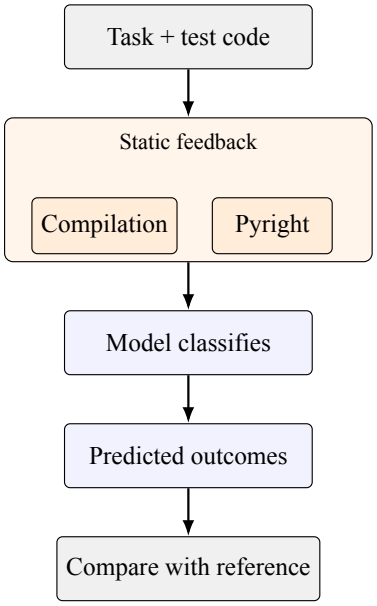


Figure 1. Tool-assisted error-detection protocol. Static analysis is supplied as supporting evidence before the model predicts the execution-based outcomes.

4.6.2 Iterative Code Generation with Static Feedback

The second tool-assisted experiment evaluates an iterative generation protocol in which static analysis feedback is provided in an iterative cycle. The model initially generates a candidate solution for the task and question. Immediately afterward, the model obtains task feedback through compilation and Pyright, using a short-circuit order; that is, it does not analyze a candidate with Pyright if it fails compilation.

Using the tool feedback, the model must choose between adjusting the code and submitting the current code. Models have up to 5 cycles to submit code; after 5 cycles, the last candidate is chosen for execution without another model decision.

It is important to remember that the feedback from static tools, which can only point out *Syntax Error* and *Runtime Errors*, and the test cases are not executed during this process. Figure 2 shows the cycle process.

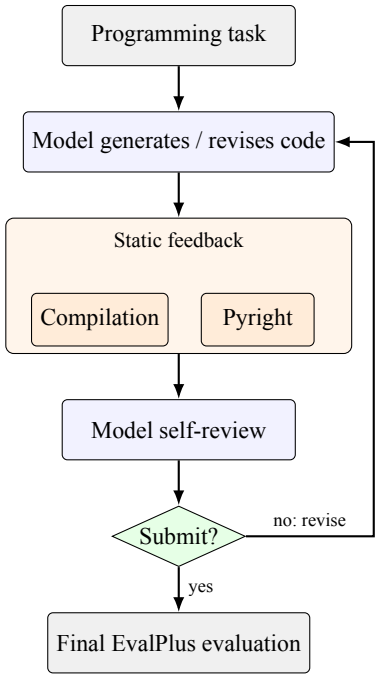


Figure 2. Tool-assisted iterative code-generation protocol. EvalPlus is applied to the final candidate and its result is not exposed to the model during generation or revision.

5 Results and Discussion

5.1 RQ1: To what extent can different language models generate correct code?

Figure 3 shows that, across all evaluated models, correct classification is the most common result, even though accuracy varies across configurations. This variation is most evident in the most notable results, such as Muse Glimmer 30B, with an accuracy of 80.8%, and Gemma 3 4B, which had the lowest accuracy at 62.2%. These results show that the SLMs can produce correct results and that different configurations yield significant differences.

Although model size is a determining factor in these differences, it is not the only one that guarantees superior performance. For example, in the Qwen 2.5 model family, the 7B configuration showed a hit rate of 73.8%, while the 32B achieved 67.1%. This demonstrates that larger models do not always outperform smaller ones in this protocol. However, this observation cannot be generalized, since the best results come from larger models, such as Muse Glimmer 30B and Gemma 4 31B.

The error distribution shows how code generation fails. Specifically, errors classified as functional are the most common, representing 15 to 30% of generations, while runtime errors are less common, at 4 to 9%. Therefore, the main difficulty lies in producing code that fully meets task requirements, rather than code that doesn't fail during execution.

Syntax error classification is rare in most models, occurring in at most 3%; however, some configurations show error patterns. The most notable case is Ornith 1.5 35B, which

Table 6. Error-detection performance by model (%). CR, FR, RR, and SR denote recall for Correct, Functional Error, Runtime Error, and Syntax Error, respectively. MF1(NS) denotes Macro-F1 computed without the Syntax Error class.

Model	Acc	MF1	CR	FR	RR	SR	MF1(NS)
General-purpose models							
Muse Glimmer 30B	73.1	53.8	83.3	55.5	27.0	52.9	61.3 (+7.5)
Gemma 4 31B	72.4	48.5	83.7	50.1	24.1	78.4	55.8 (+7.3)
Qwen 3.8 27B	62.6	42.6	62.9	75.9	12.7	64.7	48.6 (+5.9)
Qwen 2.5 Coder 7B	62.5	33.7	72.3	46.7	2.5	17.6	39.4 (+5.7)
Qwen 2.5 Coder 32B	60.8	41.0	71.1	38.1	11.4	76.5	43.3 (+2.3)
Qwen 3 Coder 30B	60.6	37.8	63.8	65.9	9.3	21.6	44.1 (+6.3)
Ornith 1.5 35B	56.3	35.7	55.4	78.1	8.0	35.3	43.1 (+7.4)
Devstral Small 2 24B	53.9	38.1	54.9	63.6	17.7	35.3	43.3 (+5.2)
GLM 4.7 Flash 30B	50.4	37.3	49.3	64.0	26.2	94.1	43.5 (+6.3)
Gemma 4 E2B IT	48.1	33.1	47.8	65.4	16.5	5.9	40.6 (+7.5)
Gemma 4 E4B IT	45.9	37.0	38.0	86.4	31.6	47.1	41.8 (+4.7)
Gemma 3 4B IT	38.5	26.7	30.0	82.0	10.1	5.9	32.4 (+5.7)
Gemma 3 4B	32.9	23.4	22.6	84.7	6.3	17.6	27.3 (+3.9)
GPT-OSS 20B	25.1	39.0	17.5	55.8	32.9	72.5	40.4 (+1.4)
Fine-tuned models							
Best Acc	77.8	50.9	85.3	69.1	27.8	11.8	63.1 (+12.2)
Weighted MF1	73.5	54.2	75.9	77.9	32.9	64.7	62.9 (+8.7)
Balanced Acc	67.8	38.1	74.4	65.8	15.2	0.0	50.9 (+12.7)
Balanced MF1	63.7	36.0	68.6	65.8	13.9	0.0	48.0 (+12.0)
Large models							
DeepSeek V4.1 Flash	75.5	51.0	89.3	51.6	14.8	35.3	54.1 (+3.1)
MiMo V2.5	51.5	39.7	47.2	79.7	21.5	96.1	44.7 (+5.0)

classifies about 10% of generations as syntax errors, 91.2% of which result from violations of the expected output structure. Furthermore, in most other models, only 20% of syntax errors are Python grammar errors. Thus, syntax errors are not limited to grammar; instead, they occur predominantly in specific patterns, according to the parameters used in the study.

In summary, the evaluated models generally produce Python code with executable structures, but the main concern is behavior that diverges from expectations, even in the most robust models that achieve accuracy close to four-fifths. Furthermore, although less frequent, syntax errors follow patterns that depend on the model used. Additionally, analysis of the Qwen 2.5 family shows that accuracy rates depend not only on model size, although it is an important factor, but primarily on configuration.

6 Threats to Validity

This study has limitations that must be considered when interpreting the results. In this section, we discuss these limitations by construct, internal and external validity.

6.1 Construct Validity

First, regarding construct validity, the experiments use fixed instruction configurations for each task performed by the LLMs, such as code generation, hallucination detection, and the interaction cycle. This choice is intentional to preserve comparability and avoid additional variation. In this way, the results characterize the model's behavior under this common instruction protocol and token set, instead of performing a prompt search that would optimize all models in general, or a specific model, but would add another body of variation that is outside the objective.

The proposed taxonomy of errors is a result of executing the generated code, encompassing both execution and test results, and can be observed through the four levels presented: *Syntax Error*, *Execution Error*, *Functional Error*, and *Correct*. However, these classifications are less detailed than those in related work, but they go beyond a binary evaluation of error and correctness, allowing broader analysis of the main errors found in the code. Furthermore, our taxonomy did not address macro-level issues already treated as non-functional requirements, such as security, performance, and maintainability, because these are not observable in the execution output.

Another taxonomy issue concerns the classification of syntax errors. Because the execution process is deterministic and fixed, malformed code, even if it deviates in ways that would cause a runtime or functional error, is marked only by the major error associated with the delusion of following the established grammar, since it is not compiled. Another relevant point about syntax execution is that, because it uses a single protocol for all languages, models with different response patterns produce text mixed with code or malformed code. We treated this as a delusion in our work, since the model deviated from the grammar requested in the prompt. This

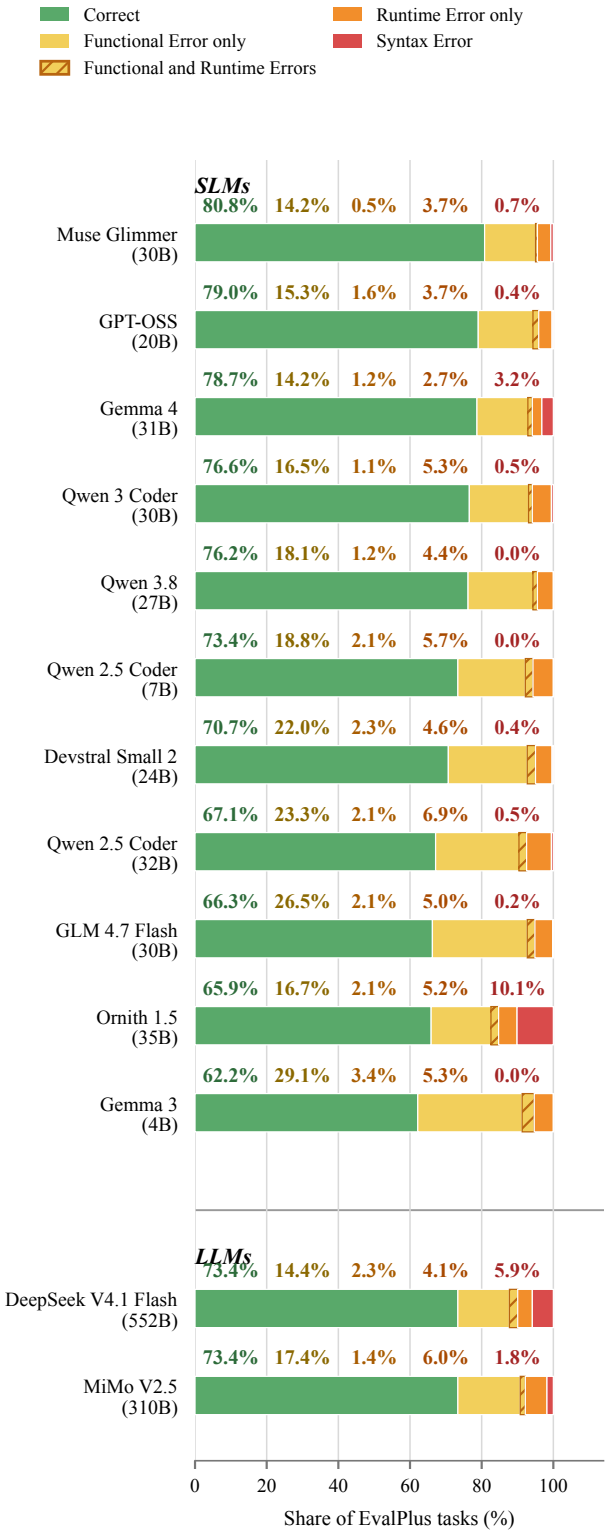


Figure 3. Code-generation outcomes across the 563 EvalPlus tasks. Each horizontal bar partitions the tasks into mutually exclusive outcomes; diagonal hatching marks programs labeled with both Functional and Runtime Errors. The five values above each bar report, respectively, Correct, Functional Error only, Functional and Runtime Errors, Runtime Error only, and Syntax Error.

was especially evident in the Ornith 1.5 35B and Gemma 4 31B models: while the other SLMs had a syntax error rate of less than 1%, these models reached 10.1% and 3.2%, respectively. In some cases, it is possible to extract the code, but to maintain the methodological rigor applied in all other experiments, we applied the error label.

Finally, the label *Correct* is determined only from the available test sets of EvalPlus, not from a real proof that the model will have the expected behavior for all possible inputs, so it does not serve as a formal proof of semantic correctness.

6.2 Internal Validity

Another limitation concerns the models used in the comparisons. The local SLMs used for the tasks were mostly quantized. We adopted this approach based on prior empirical work showing no significant loss when quantizing models to 4 bits and that, with quantization, we could use larger models given hardware limitations. To maintain methodological rigor, we used all models in quantized form in the comparison, even those that would fit the computation without precision reduction. In one model used for comparison, we noted that the unquantized variant performed slightly better than the quantized version. This difference does not invalidate these models; however, model performance should not be interpreted as the total performance of the model without precision reduction. Furthermore, this result shows that larger quantized models may be more advantageous than smaller lossless models.

In our LLM experiments, we performed inference from a single generation rather than from multiple responses to the same input, since LLMs behave non-deterministically and responses may vary, although we applied procedures to stabilize their behavior. Therefore, future work could investigate generating multiple outputs for a given task and, through statistical selection, choose the most frequent outputs as the final result.

Another limitation concerns the models selected for specialization, since we evaluated only Gemma models that fit our hardware limitations. The improvements show that specialization can be effective in the evaluated scenario, but this pattern may not hold for other models or adaptation methods. Furthermore, limitations included the limited range of hyperparameters, the number of runs, and the optimization targets. Future work could expand the scope of hyperparameters by conducting more searches.

The distillation process can introduce noise into the supervision and should not be treated as entirely correct, as the model's explanations may ignore error cases in the code or present false or inaccurate information. However, the introduction of rationales also builds on prior work showing that rationales can improve the model's generalization, rather than simply making it predict a class.

Another limitation lies in the feedback given to the models, because, as seen in RQ4, Pyright is not a source of absolute truth like the language compiler, and therefore, it can provide feedback that may be detrimental to the model, as seen in the results of RQ5, where some models had lower performance in recalling the detection of the *Runtime Error* class. However, even with a warning that the result would

not necessarily be correct.

6.3 External Validity

The experiments are limited to Python and do not imply the same results for the same task in other languages. Furthermore, the benchmark used only contains isolated tasks at the function level and does not reflect more robust behavior in real-world programming scenarios, such as those involving repositories. This benchmark is used to conduct a study. Initially, to perform a direct comparison with works that have already mapped the main errors in the same tasks, to show the current state of the models and how the difference between larger and smaller models has changed throughout the analyses. Another motivation for using this dataset, rather than real repositories, is that models would need to perform additional skills, such as environment interactions and agent studies, during the execution cycle, which would fall outside the task's overall scope.

The nominal parameter count should be considered only as an indicator of difference, and not as the only absolute difference between the models. Although some models of the same version, with variations in size, may have similar internal factors, there are still other factors that differ between models, in addition to size, such as the training dataset, the internal architecture during all phases related to training, among other factors that cannot be controlled internally.

7 Conclusion

This study investigated SLMs ability to generate code and analyzed how many test cases the model passes by verifying the execution of the generated code, to better understand the main types of errors in these models. We conducted experiments to verify whether SLMs are capable of generating correct code, what the most frequent errors are based on the degree of error, the ability of the models to identify these errors in the code, how specialization can improve this task, and how the models can be compared with static analyses and what the impact of these analyses is on the evaluation and generation of code by the models.

The code-generation results show that SLMs can generate valid code, producing operational code in more than 90% of generations and reaching almost 80% fully functional generation. However, the main error of the models lies in the inability to understand all the nuances of functional requests, which leads to deviations from the requested tasks, resulting in a *Functional Error*. The least frequent type of hallucination is *Syntax Error*, which highlights the model's difficulty in following the language's grammar. In one case, there were many errors; however, these were not due to code errors but to the model using natural language instead of the requested language. These results corroborate previous work pointing to this behavior; however, this experiment shows that models still struggle with it, although they reduce it across all hallucination types. This analysis also used feedback generated by static tools, which reduced the number of codes with *Functional Error*; however, it affected other classes, including those that should not be affected by the feedback, and some performed worse on the compilable code response.

The results on the models' ability to detect code hallucinations, using a process called LLM-as-a-judge, show a divergence between code generation and evaluation, since models that perform well at code generation are not necessarily good at code review. The main difficulty with these models is identifying Runtime Errors, meaning that while they can generate code with correct execution flows, they may not detect certain problems. However, compared with static tools, which can detect runtime errors, SLM analysis is, in most cases, superior.

A strength of some models was their ability to detect functional errors in the analyzed code, demonstrating an interesting skill as reviewers of requested functionality, something static tools cannot measure, making these local models a promising use case. However, although the models can identify these flaws, the correction is not always effective; although the models that managed to self-detect managed to improve the accuracy rate, the conversion was not total.

Using static tools in the cycle, along with a self-evaluation mechanism, allowed some models to correct invalid writing errors and, in all cases, runtime errors. However, in some models, iterativity was less stable, with cases of regression. Regression is also evident when the task is limited to evaluating code for runtime errors. Because Pyright performs worse than the models, the models began to treat its output as absolute truth and, even with the warning not to, ended up relying on static analysis and missing some cases. The effective use of the added context was for *Syntax Error*, as most models managed to have complete detection in most scenarios with this feedback, which reinforces that, although the models do not have the capabilities to detect syntax errors during reading, integration with other tools can be useful, especially within an iterative cycle, so that the model provides code free of this type of error.

Finally, the specialized hallucination-detection model, while showing superior results compared to smaller traditional models, still performs significantly worse. Some considerations relate to the training target: due to the unbalanced dataset, the trained models showed low capacity to detect lower-representation models and demonstrated a bias toward selecting the majority class during training.

7.1 Future Work

The most important direction for future work is to evaluate how these models behave in environments that replicate real-world software development. In real-world development, logic isn't tied to a single function; it spans the entire work environment, including files, external APIs, dependencies, local requirements, and specific execution requirements. A natural extension is the use of benchmarks that investigate at the repository level. However, some problems don't map as effectively as in the three classes we've presented. Adding the complexity of repository-level work introduces other considerations, such as context control and engineering, and models' ability to use tools in an agentic loop. These are variables that should be considered in a later study.

Another advantage to leverage in real-world development environments is verifying other runtime problems. In other

analyzed works, many errors consisted of hallucinations of repository symbols, often due to a lack of context. By adding real-world development environments and using an agentic, interactive cycle, it's possible to observe more deeply what types of execution errors models commit, even when the context includes the symbol to be used.

Other works can perform less complex operations, which would expand the current analysis by measuring the effect of using these models without quantization on these results. Furthermore, for model training, it is interesting to expand the use of variable elements, from using other models for training to using a larger training set. However, this approach can be computationally very costly and time-consuming.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.

Andriushchenko, G., Garaev, R., Rvanova, L., Shelmanov, A., and Ivanov, V. V. (2026). Efficient hallucination detection in automatic code generation. In Liakata, M., Moreira, V. P., Zhang, J., and Jurgens, D., editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 43197–43220, San Diego, California, United States. Association for Computational Linguistics.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. (2021). Program synthesis with large language models.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code.

DeepSeek-AI (2026). DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report.

Dou, S., Jia, H., Wu, S., Zheng, H., Wu, M., Tao, Y., Zhang, M., Chai, M., Fan, J., Xi, Z., Zheng, R., Wu, Y., Wen, M., Gui, T., Zhang, Q., Qiu, X., and Huang, X. (2025). What's wrong with your code generated by large language models? an extensive study. Accepted by Science China Information Sciences, according to the arXiv record.

Gemma Team, El Abd, S., Aggarwal, V., Algayres, R., Andreev, A., Bachem, O., et al. (2026). Gemma 4 Technical Report.

Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., et al. (2025). Gemma 3 Technical Report.

Giagnorio, A., Mastropaolo, A., Afrin, S., Di Penta, M., and Bavota, G. (2026). Evaluating the impact of post-training quantization on large language models for code generation. In *Proceedings of the 2026 34th IEEE/ACM International Conference on Program Comprehension*, ICPC '26, pages 195–206, New York, NY, USA. Association for Computing Machinery.

GLM-4.5 Team, Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., et al. (2025). GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models.

Hsieh, C.-Y., Li, C.-L., Yeh, C.-k., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada. Association for Computational Linguistics.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., et al. (2024). Qwen2.5-Coder Technical Report.

Lee, Y., Song, J. Y., Kim, D., Kim, J., Kim, M., and Nam, J. (2025). Hallucination by code generation LLMs: Taxonomy, benchmarks, mitigation, and challenges.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc.

Mahade Hasan, M., Waseem, M., Kemell, K.-K., Rasku, J., Ala-Rantala, J., and Abrahamsson, P. (2026). Assessing small language models for code generation: An empirical study with benchmarks. *Journal of Systems and Software*, 236:112815.

Meta Superintelligence Lab (2026). Muse Glimmer 30B Model Card. Hugging Face model card.

Microsoft (2026). Pyright: Static type checker for python. Software repository.

Mistral AI (2025). Introducing Devstral 2 and Mistral Vibe CLI. Official model release.

Nguyen, K., Ton, K., Phan, N., Khalil, I., Tran, K., Borcea, C., Jin, R., Khreishah, A., and Thai, M. T. (2026). Noir: Privacy-preserving generation of code with open-source llms.

OpenAI, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., et al. (2025). gpt-oss-120b & gpt-oss-20b model card.

Ornith Team (2026). Ornith-1.5: From self-scaffolding to self-improvement. Official model report.

Pavel, M., Petrovic, N., Mazur, L., Zolfaghari, V., Pan, F., and Knoll, A. (2025). Hallucination in LLM-based code generation: An automotive case study.

Python Software Foundation (2026). Built-in function `compile`. Python 3.13 documentation.

Qwen Team (2025). Qwen3-Coder: Agentic coding in the world. Official model release.

Qwen Team (2026). Qwen3.8-Max: A new bar for coding and cowork. Official model release.

Rastogi, A., Yang, A., Jiang, A. Q., Liu, A. H., Sablayrolles, A., et al. (2025). Devstral: Fine-tuning language models for coding agent applications.

Shen, L., Yang, Q., Huang, X., Ma, Z., and Zheng, Y. (2025). Gpiot: Tailoring small language models for iot program synthesis and development. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems*, SenSys '25, pages 199–212, New York, NY, USA. Association for Computing Machinery.

Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., and Song, D. (2025). CodeHalu: Investigating code hallucinations in LLMs via execution-based

verification. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, pages 25300–25308. AAAI Press.

Xiaomi MiMo Team (2026). MiMo-V2.5. Official model card.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., et al. (2025). Qwen3 Technical Report.

Z.ai (2025). GLM-4.7-Flash model card. Hugging Face model card.